Sirus AI CRM — 系统概要设计

> 文档版本: v0.4
> 更新日期: 2026-03-03
> 设计理念: **AI-Native CRM** — Agent 不是 CRM 的附加层，Agent 就是 CRM 本身
> v0.4 变更: §13 重写为全自动化开发流水线模式（Orchestrator + aider 替代人工 Copilot 操作）
> v0.5 变更: §6.1 + §13 移除 vLLM B，Orchestrator 重新定位为薄调度层（aider 已覆盖核心编码循环）

1. 设计输入

文档	说明
01-基础需求-sirus项目需求.md	基础 CRM 功能需求，15 个一级模块
02-智能扩展需求.md	AI 智能扩展需求，13 个智能化模块
03-硬件清单.md	四节点硬件清单

2. 核心理念：AI-Native CRM

2.1 传统 CRM vs AI-Native CRM

维度	传统 CRM	AI-Native CRM (本方案)
交互方式	人操作表单/菜单	人与 Agent 对话，Agent 执行一切
数据录入	人手动填写字段	Agent 从对话/邮件/通话中自动提取
业务流程	硬编码工作流 (if-else)	Agent 基于上下文动态决策
规则引擎	预定义规则触发	Agent 理解业务语义，自主判断
报表分析	预设报表模板	Agent 实时分析，自然语言问答
通知提醒	基于规则的定时触发	Agent 感知业务变化，主动推送
客户画像	标签体系 + 手动标注	Agent 持续理解客户，动态生成
销售辅导	经理人工指导	Agent 分析行为模式，个性化建议
系统集成	API 对接 + ETL	Agent 理解跨系统语义，智能编排
用户界面	复杂的 Web 应用	钉钉对话(主) + 轻量看板(辅)

2.2 一句话定位

> **Sirus 不是一个带 AI 功能的 CRM，而是一个理解销售业务的 AI Agent，它的记忆就是 CRM 数据。**

2.3 设计原则

原则	说明
Agent-First	每个 CRM 功能首先设计为 Agent 能力，而非表单界面

原则	说明
对话即操作	用户通过自然语言完成所有 CRM 操作，零学习成本
感知即录入	Agent 从交互中自动感知并记录信息，而非等人填写
理解即流程	Agent 理解业务语义后自主推进流程，无需硬编码工作流
记忆即数据	结构化数据库是 Agent 的长期记忆，Agent 自主读写
主动即服务	Agent 不需要等待指令，主动发现问题、推送洞察、建议行动
可降级	AI 不可用时，基础 CRUD 仍通过 Web 看板可操作

3. 架构总览

3.1 AI-Native 分层架构

```
┌──────────────
```

```
│   │ 结构化记忆    │ │ 语义记忆      │ │ 工作记忆              │   │
│   │ PostgreSQL   │ │ 向量数据库    │ │ Redis               │   │
│   │              │ │              │ │                     │   │
│   │ · 客户/商机  │ │ · 赢单经验   │ │ · 对话上下文          │   │
│   │ · 合同/订单  │ │ · 行业知识   │ │ · 任务执行状态        │   │
│   │ · 活动记录   │ │ · 产品知识   │ │ · 待确认操作          │   │
│   │ · 审计事实   │ │ · 客户洞察   │ │ · 会话变量            │   │
│   │ · 用户权限   │ │ · FAQ/话术   │ │ · Agent 推理链        │   │
│   └──────────────┘ └──────────────┘ └──────────────────────┘   │
│                                                                 │
├─────────────────────────────────────────────────────────────────┤
│                                                                 │
│                        事件总线 (Agent 的神经网络)               │
│                                                                 │
│   所有业务操作 → 产生事件 → Agent 订阅 → 触发主动行为            │
│   (Redis Stream / PostgreSQL LISTEN-NOTIFY)                     │
│                                                                 │
│   事件类型:                                                      │
│   · 数据变更事件 (create/update/delete)                          │
│   · 时间触发事件 (每日晨报/周报/月度复盘)                         │
│   · 阈值突破事件 (线索衰减/商机停滞/回款逾期)                     │
│   · 外部输入事件 (钉钉消息/邮件/Webhook)                         │
│   · Agent 协作事件 (Agent间任务委派)                             │
│                                                                 │
└─────────────────────────────────────────────────────────────────┘
```

3.2 与传统架构的关键区别

```
传统 CRM 架构:
  用户 → Web UI → 业务服务层 → 数据库
                      ↕
                  AI 辅助层 (可有可无)

AI-Native 架构 (本方案):
  用户 → 钉钉对话 → Agent 大脑 → 工具调用 → 记忆读写
                      ↕              ↕
                   事件总线       外部系统
                      ↕
                  主动推送 → 用户
```

核心变化：没有「业务服务层」——Agent 大脑就是业务逻辑。传统 CRM 里用代码写的 if-else 流程，在这里由 Agent 理解语义后动态决策。

4. CRM 功能的 AI 化实现

> 传统 CRM 的每个模块，不再是「界面 + 后端逻辑 + 数据库」，而是「Agent 能力 + 工具 + 记忆」。

4.1 总览：CRM 模块 → Agent 能力映射

传统 CRM 模块	AI-Native 实现方式	触发方式
线索管理	Agent 自动采集/评分/分配/跟进编排	事件驱动 + 对话
客户管理	Agent 持续构建客户认知，动态维护画像	事件驱动 + 定时
联系人管理	Agent 自动识别决策链，维护关系图谱	数据变更事件
商机管理	Agent 全程跟踪/预测赢率/推荐行动/检测卡点	事件驱动 + 对话

传统 CRM 模块	AI-Native 实现方式	触发方式
产品管理	Agent 管理产品知识库，智能推荐产品组合	对话 + 知识检索
合同管理	Agent 审查条款/预检/到期追踪	事件驱动 + 定时
订单管理	Agent 跟踪履约/异常检测/进度播报	外部系统事件
回款管理	Agent 追踪回款/预警逾期/推荐催收策略	定时 + 阈值事件
目标管理	Agent 智能拆解/实时追踪/差距分析/行动建议	定时 + 对话
行为管理	Agent 自动记录/分析模式/生成日报周报	事件驱动 + 定时
数据分析	Agent 自然语言问答/主动推送洞察/预测	对话 + 定时
钉钉集成	Agent 通过钉钉与人交互（这就是界面）	全时在线
审批流程	Agent 在钉钉内完成全流程审批	事件驱动
系统集成	Agent 理解跨系统语义，智能编排调用	事件驱动

4.2 示例：传统流程 vs AI-Native 流程

示例一：新建线索

```
传统 CRM:
  销售 → 打开CRM → 点击"新建线索" → 填写15个字段 → 保存
  (耗时10分钟，可能漏填)

AI-Native:
  销售在钉钉说："刚在展会认识了一个做教育的客户，张总，
                 电话13800138000，他们想采购一批机械臂用于教学"
  Agent:
    1. 理解意图 → 创建线索
    2. 提取实体 → 姓名:张总，电话:13800138000，行业:教育，需求:机械臂教学
    3. 调用天眼查工具 → 自动补全企业信息
    4. 调用线索评分工具 → 计算评分 78分 (教育行业+明确需求)
    5. 调用线索CRUD工具 → 写入数据库
    6. 调用钉钉工具 → 推送确认卡片给销售
    7. 产生事件 → 触发分配Agent分析最优销售
  (耗时15秒，信息完整)
```

示例二：商机阶段推进

```
传统 CRM:
  销售 → 打开CRM → 找到商机 → 手动改阶段 → 填写推进记录 → 保存
  经理 → 每周打开漏斗报表 → 人工分析哪些商机卡住了

AI-Native:
  销售在钉钉说："今天给XX客户做了产品演示，反馈很好，他们下周出方案评审"
  Agent:
    1. 理解：商机推进 → 产品演示完成 → 客户正面反馈 → 即将方案评审
    2. 自动更新商机阶段：需求确认 → 方案评审
    3. 自动记录拜访活动 + AI摘要
    4. 重新计算赢率：45% → 62%（正面反馈 + 推进顺利）
    5. 推荐下一步："建议准备技术方案，安排售前工程师下周一前完成"
    6. 推送给经理："XX商机推进到方案评审阶段，赢率提升至62%"

  与此同时（主动型Agent):
    巡检Agent每日扫描："YY商机在需求确认阶段已停留22天(平均12天)，
    建议联系客户确认是否需要调整方案" → 推送给负责销售
```

示例三：回款管理

```
传统 CRM:
  财务 → 手动核对银行流水 → 手动标记回款 → 手动通知销售

AI-Native:
  Agent(主动型):
    1. 每日检查应收账款计划
    2. 发现某笔50万即将到期(3天后)
    3. 查询客户历史回款习惯：平均延迟5天
    4. 查询客户最新状态：健康度评分降了10分
    5. 综合判断：中等逾期风险
    6. 推送给客户经理："XX客户50万回款3天后到期，历史有延迟倾向，
       建议今天电话提醒。参考话术：..."
    7. 如果逾期7天，自动升级：推送给销售经理 + 建议催收策略
```

4.3 CRM 功能的三种 AI 化模式

模式	说明	适用场景	举例
对话即操作	用户说一句话，Agent 完成完整操作	所有 CRUD 操作	"帮我创建一个线索" "把这个商机推到下一阶段"
感知即记录	Agent 从对话/ 事件中自动提取信息并记录	拜访记录、跟进记录、 客户信息更新	销售聊天内容中提到客户新需求 → 自动更新客户画像
主动即服务	Agent 不等指令，基于事件/ 时间主动行动	预警、评分、规划、报告	晨间推送、线索衰减预警、季度复盘

5. Agent 架构详细设计

5.1 Supervisor Agent (总调度)

Supervisor 是 Agent 大脑的中枢，负责：

```
用户消息 / 系统事件
        │
        ▼
┌──────────────────────────────────────┐
│         Supervisor Agent             │
│                                      │
│  ① 意图理解                          │
│     "用户想干什么？这个事件意味着什么？"  │
│                                      │
│  ② 上下文加载                        │
│     从记忆层加载相关信息              │
│     （当前对话历史、用户角色、相关业务数据）│
│                                      │
│  ③ 任务规划                          │
│     "需要做哪些步骤？调用哪些工具？"    │
│     "需要委派给哪个专业 Agent？"       │
│                                      │
│  ④ 执行编排                          │
│     按计划顺序/并行执行工具调用        │
│     处理中间结果、判断是否需要追加步骤  │
│                                      │
│  ⑤ 风险评估                          │
│     "这个操作是否需要人确认？"         │
│     低风险→自动执行 / 高风险→请求确认   │
│                                      │
│  ⑥ 结果编排                          │
│     将执行结果组织为自然语言回复       │
│     决定推送渠道和形式                 │
│                                      │
└──────────────────────────────────────┘
```

5.2 Agent 类型与职责

响应型 Agent (对话触发)

Agent	能力	可用工具
销售助手 Agent	理解销售的自然语言指令，执行CRM操作	全部CRM CRUD工具、钉钉工具
数据问答 Agent	理解管理层的数据问题，查询分析并回答	报表查询工具、统计聚合工具、趋势分析工具
合同审查 Agent	理解合同条款，对比模板，标记风险	合同模板工具、风险评估工具

主动型 Agent (事件/定时触发)

Agent	能力	触发条件
巡检 Agent	扫描全部业务数据，发现异常和遗漏	每日定时
预警 Agent	监控阈值突破，推送分级预警	数据变更事件 + 定时检查
规划 Agent	为每位销售编排工作计划	每日晨间 + 周一
催收 Agent	追踪应收账款，分级催收策略	每日定时 + 逾期事件
晨报 Agent	生成个性化每日简报	每日 08:00

分析型 Agent (数据驱动)

Agent	能力	输出
客户洞察 Agent	构建/更新客户画像，评估健康度	客户画像JSON + 风险提示
商机预测 Agent	预测赢率，分析卡点，推荐行动	赢率分数 + 推进建议
漏斗分析 Agent	分析销售漏斗健康度，预测收入	漏斗报告 + 达标预测
行为分析 Agent	分析销售行为模式，提炼最佳实践	行为报告 + 辅导建议
线索评分 Agent	多维度评估线索质量	0-100分 + 评分依据

5.3 Agent 协作模式

```
场景：新线索进入系统

外部事件：展会扫码表单提交
    │
    ▼
Supervisor Agent
    │
    ├─► 线索评分 Agent
    │     └─ 使用：评分计算工具、天眼查工具
    │     └─ 输出：评分 82，高质量
    │
    ├─► 客户洞察 Agent
    │     └─ 使用：天眼查工具、行业知识检索
    │     └─ 输出：企业画像 + 潜在需求分析
    │
    └─► 规划 Agent（基于评分结果）
          └─ 使用：数据查询工具(销售负载)、匹配算法
          └─ 输出：推荐分配给销售A(行业匹配+低负载)
    │
    ▼
Supervisor 汇总：
    1. 写入线索（CRM CRUD工具）
    2. 推送到钉钉（互动卡片）：
       "新高质量线索！XX教育科技，评分82分
        行业：教育 | 规模：200人 | 需求：教学机械臂
        建议分配给：销售A（教育行业经验丰富）
        [领取] [查看详情] [转给他人]"
```

5.4 工具系统设计

每个工具是一个标准化的 Python 函数，Agent 通过 Tool Calling 机制调用：

```python
# 工具定义示例
class Tool:
    name: str           # "create_lead"
    description: str    # "创建销售线索，需要提供联系人信息和来源"
    parameters: dict    # JSON Schema 定义参数
    risk_level: str     # "low" | "medium" | "high"
    requires_confirm: bool  # 是否需要人确认

# 工具分类
tools = {
    # CRM 数据工具（直接操作数据库）
    "crm_data": [
        "create_lead", "update_lead", "query_leads", "delete_lead",
        "create_customer", "update_customer", "query_customers",
        "create_opportunity", "update_opportunity", "query_opportunities",
        "create_activity", "query_activities",
        "create_contract", "update_contract", "query_contracts",
        "create_order", "query_orders",
        "query_payments", "create_payment_record",
        "query_targets", "update_target_progress",
    ],

    # 钉钉通讯工具
    "dingtalk": [
        "send_message", "send_interactive_card", "create_approval",
        "create_group", "add_group_member", "send_group_message",
        "get_user_info", "sync_org_structure",
    ],

    # 外部数据工具
    "external": [
        "query_company_info",       # 天眼查/企查查
        "query_bid_announcements",  # 招标信息
        "query_industry_news",      # 行业资讯
        "query_competitor_info",    # 竞品信息
    ],

    # 计算分析工具
    "analytics": [
        "calculate_lead_score",     # 线索评分
        "predict_win_rate",         # 赢率预测
        "analyze_funnel",           # 漏斗分析
        "generate_forecast",        # 收入预测
        "analyze_behavior_pattern", # 行为模式分析
        "calculate_health_score",   # 客户健康度
        "detect_anomaly",           # 异常检测
    ],

    # 文档生成工具
    "document": [
        "generate_daily_report",    # 日报生成
        "generate_weekly_report",   # 周报生成
        "generate_review_report",   # 复盘报告
        "generate_talk_script",     # 话术推荐
        "review_contract_terms",    # 合同条款审查
    ],

    # 系统集成工具（二期）
    "integration": [
        "erp_query_inventory",      # ERP库存查询
```

```
        "erp_query_production",     # ERP生产进度
        "mes_query_status",         # MES制造状态
        "fenbeibei_create_trip",    # 分贝通差旅
    ],
}
```

5.5 记忆系统设计

```
┌──────────────────────────────────────────────────────────────┐
│                      Agent 记忆系统                          │
│                                                              │
│  ┌──────────────────────────────────────────────────────┐    │
│  │              工作记忆 (Redis)                        │    │
│  │  存活时间: 分钟~小时级                                │    │
│  │                                                      │    │
│  │  · 当前对话上下文 (最近 N 轮)                         │    │
│  │  · 待确认操作队列 (写操作二次确认)                    │    │
│  │  · Agent 推理链缓存 (中间步骤)                        │    │
│  │  · 用户会话状态 (正在编辑的实体等)                    │    │
│  └──────────────────────────────────────────────────────┘    │
│                                                              │
│  ┌──────────────────────────────────────────────────────┐    │
│  │           结构化记忆 (PostgreSQL)                    │    │
│  │  存活时间: 永久                                       │    │
│  │                                                      │    │
│  │  · 所有 CRM 业务数据 (客户/商机/合同/活动...)         │    │
│  │  · 用户信息 & 权限 (钉钉同步)                         │    │
│  │  · 审计事实流 (不可变事件日志)                        │    │
│  │  · Agent 操作日志 (who/what/when/why)                 │    │
│  │  · 配置 & 规则 (风险等级、评分权重等)                 │    │
│  └──────────────────────────────────────────────────────┘    │
│                                                              │
│  ┌──────────────────────────────────────────────────────┐    │
│  │             语义记忆 (向量数据库)                    │    │
│  │  存活时间: 永久, 持续增长                             │    │
│  │                                                      │    │
│  │  · 赢单/丢单经验库 (历史案例向量化)                   │    │
│  │  · 产品知识库 (产品文档、FAQ)                         │    │
│  │  · 行业知识 (行业特征、季节规律)                      │    │
│  │  · 客户深度洞察 (Agent 分析结论缓存)                  │    │
│  │  · 销售话术库 (成功案例话术模板)                      │    │
│  │  · 企业规章 & CRM 使用规范                            │    │
│  └──────────────────────────────────────────────────────┘    │
│                                                              │
│  记忆调用策略:                                               │
│  Agent 收到任务时 →                                          │
│    1. 工作记忆: 加载对话上下文 (必须)                        │
│    2. 结构化记忆: 查询相关业务数据 (按需)                    │
│    3. 语义记忆: 检索相似经验/知识 (按需)                     │
└──────────────────────────────────────────────────────────────┘
```

6. 物理部署架构

基于五节点内网硬件（含 Orchestrator 专用服务器，共 5 台，全部内网）：

```
┌──────────────────────────────────────────────────────────────────────────────┐
│                              内部网络 172.16.x.x                              │
│                                                                              │
│                        ┌──── 网关服务器 (172.16.14.215) ────┐                 │
│                        │ OS: Ubuntu 24.04 | i5-220H | 32G   │                 │
│     外部流量 ──────>│                                    │                 │
│    (钉钉/浏览器)  │  ┌──────────────────────────┐    │                 │
│                        │  │ Nginx 反向代理              │    │                 │
│                        │  │ - 统一入口 :80/:443         │    │                 │
│                        │  │ - SSL 终结                  │    │                 │
│                        │  │ - 路由分发                  │    │                 │
│                        │  │ - 限流 / 访问控制           │    │                 │
│                        │  └──────────────────────────┘    │                 │
│                        │  ┌──────────────────────────┐    │                 │
│                        │  │ 钉钉回调接收                │    │                 │
│                        │  │ - 机器人 Webhook 接入       │    │                 │
│                        │  │ - 事件订阅回调              │    │                 │
│                        │  └──────────────────────────┘    │                 │
│                        └──────────────────────────────────┘                 │
│                                 │ 反向代理                                    │
│                        ┌────────┼────────┐                                   │
│                        ▼        ▼        ▼                                   │
│  ┌──── 边缘CRM (172.16.12.50) ────┐  ┌── AI算力 (172.16.11.194) ──┐          │
│  │ OS: macOS 15.1 | M4 Pro | 64G  │  │ OS: Ubuntu 22.04           │          │
│  │                                │  │ 2×RTX4090 | 128G           │          │
│  │ ┌──────────────────────────┐   │  │                            │          │
│  │ │ CRM 后端 (FastAPI)        │   │  │ ┌──────────────────────┐ │          │
│  │ │ - 业务 API        :8900   │   │  │ │ vLLM 推理服务         │ │          │
│  │ └──────────────────────────┘   │  │ │ - Qwen3-30B-A3B  :8000 │ │          │
│  │ ┌──────────────────────────┐   │  │ └──────────────────────┘ │          │
│  │ │ CRM 前端 (Vue3)           │   │  │ ┌──────────────────────┐ │          │
│  │ │ - Web UI          :3000   │   │  │ │ AI Agent 引擎         │ │          │
│  │ └──────────────────────────┘   │  │ │ - 调度/编排      :8100 │ │          │
│  │ ┌──────────────────────────┐   │  │ └──────────────────────┘ │          │
│  │ │ PostgreSQL        :5432   │   │  └──────────────────────────┘          │
│  │ └──────────────────────────┘   │                                         │
│  │ ┌──────────────────────────┐   │                                         │
│  │ │ Redis             :6379   │   │                                         │
│  │ └──────────────────────────┘   │                                         │
│  └────────────────────────────────┘                                         │
│                        ▲                                                    │
│                        │ TCP (备份/同步)                                     │
│                        ▼                                                    │
│  ┌──── 数据中心 (172.16.14.90) ────┐                                        │
│  │ OS: Ubuntu 22.04 | i5-220H | 32G │                                      │
│  │                                  │                                       │
│  │  ┌──────────────────────────┐    │                                       │
│  │  │ PostgreSQL 热备           │    │                                       │
│  │  │ - 流复制 Standby    :5432 │    │                                       │
│  │  └──────────────────────────┘    │                                       │
│  │  ┌──────────────────────────┐    │                                       │
│  │  │ 监控与日志                 │    │                                       │
│  │  │ - Prometheus + Grafana    │    │                                       │
│  │  │ - 日志归集                 │    │                                       │
│  │  └──────────────────────────┘    │                                       │
│  │  ┌──────────────────────────┐    │                                       │
│  │  │ 异步任务 Worker            │    │                                       │
│  │  │ - Celery Worker           │    │                                       │
│  │  │ - 定时任务 (巡检/报告)      │    │                                       │
│  │  │ - 数据同步任务             │    │                                       │
```

```
│  │  └──────────────────────────┘     │                                  │
│  │  ┌──────────────────────────┐     │                                  │
│  │  │ 文件/知识库备份                  │     │                                  │
│  │  └──────────────────────────┘     │                                  │
│  └──────────────────────────────────┘                                  │
└──────────────────────────────────────────────────────────────────────┘
```

6.1 节点职责与部署决策

节点	角色	部署内容	决策理由
网关服务器 (172.16.14.215)	流量入口	Nginx 反向代理、SSL 终结、钉钉回调接收	独立网关隔离外部流量，内部服务不直接暴露；16核32G足以承担代理和限流
边缘CRM设备 (172.16.12.50)	业务主机	CRM 后端 (FastAPI)、前端 (Vue3)、PostgreSQL 主库、Redis	M4 Pro 12核+64G 性能充裕；7.3TB 存储放数据库绑绑有余；业务与数据同机降低延迟
AI算力中心 (172.16.11.194)	AI 推理	vLLM (Qwen3-30B-A3B)、Agent 引擎	2×4090 共 98GB 显存，专用于 LLM 推理；Agent 引擎与 LLM 同机，调用零网络延迟
数据中心 (172.16.14.90)	数据保障	PostgreSQL 热备、监控 (Prometheus+Grafana)、Celery Worker、日志归集、文件备份	独立节点做数据冗余和异步计算，CRM 主机故障时数据不丢失；32G 适合跑后台任务
Orchestrator 专用服务器 (172.16.14.201)	调度中心 + Staging	Orchestrator 薄调度层（基于 aider 已有编码循环，负责跨机器任务编排→SSH 分发→结果汇总→通知）、**pytest 本地执行**、云端 API (AutoReview/DocParser)、云端API多模型 Judge、**预发布验收 (Staging)**（独立 Staging DB/配置）	Xeon 64C + 128G + 2×4090，专机专用保证 7×24 调度稳定性；vLLM B 已移除，AutoReview/DocParser 改用云端 API；预发布验收使用独立数据库+配置与开发环境隔离；Agent 推理统一调用 4090 vLLM A

> **v0.5 变更**: aliyun-agent（云端 L20）已从架构中移除，全部业务在内网 5 台机器运行。预发布验收由 orchestrator 承担，vLLM C 取消。

6.2 网络流量路径

```
外部请求（钉钉/浏览器）
        │
        ▼
  网关服务器（.14.215）
  Nginx :80/:443
        │
        ├── /api/*  ──────> 边缘CRM（.12.50:8900）  CRM后端
        ├── /       ──────> 边缘CRM（.12.50:3000）   前端页面
        └── /dingtalk/* ──> 边缘CRM（.12.50:8900）  钉钉回调

  CRM后端（.12.50）
        │
        ├── AI请求 ──────> AI算力（.11.194:8100）    Agent引擎
        ├── DB读写 ──────> 本机 PostgreSQL（:5432）   主库
        └── 缓存 ────────> 本机 Redis（:6379）

  Agent引擎（.11.194:8100）
        │
        └── LLM调用 ─────> 本机 vLLM（:8000）

  数据中心（.14.90）
        │
        ├── 流复制 <──────── 边缘CRM PostgreSQL 主库
        ├── 日志归集 <────── 各节点日志
        └── Celery Worker —> 连接 边缘CRM Redis（:6379）作为 Broker
```

7. 技术选型

> AI-Native 架构下，技术选型围绕「Agent 运行环境」组织，而非传统的前后端分层。

7.1 Agent 运行时 (核心)

组件	选型	AI-Native 角色
LLM 推理引擎	**vLLM**	Agent 的「思考能力」，高性能推理，PagedAttention，OpenAI 兼容 API
LLM 模型	**Qwen3-30B-A3B** (一期)	MoE 架构，激活 3B 参数，推理快，2×4090 可部署；后续可升级 Qwen3.5-35B-A3B
Agent 框架	**自研轻量框架**	基于 vLLM Tool-Calling 原生能力 + Prompt 模板引擎，不引入 LangChain 等重型框架
LLM 客户端	**OpenAI SDK** (Python)	vLLM 兼容 OpenAI API，标准化接口，模型可切换
Prompt 管理	**Jinja2 模板 + Git 版本控制**	Prompt 即代码，支持 A/B 测试和版本回滚
向量数据库	**pgvector** (PostgreSQL 扩展)	Agent 的语义记忆，不额外引入独立向量数据库，降低运维复杂度
Embedding 模型	**一期：阿里云百炼 Embedding API** (text-embedding-v3) 二期：本地 **BGE-M3**（4090 与 vLLM A 并存）	中文语义理解，用于知识库检索和经验匹配。一期知识量小走云端 API（零部署、成本~几元/月），二期知识量上来后迁本地

7.2 记忆层 (数据存储)

组件	选型	AI-Native 角色
结构化记忆	**PostgreSQL 16** + pgvector	Agent 的长期记忆：CRM 业务数据 + 向量索引（一库两用）
工作记忆	**Redis 7**	Agent 的短期记忆：对话上下文、待确认队列、推理链缓存
事件总线	**Redis Stream**	Agent 的神经网络：业务事件发布/订阅，驱动主动型 Agent
ORM	**SQLAlchemy 2.0** (async)	工具层访问结构化记忆的接口
数据库迁移	**Alembic**	记忆结构的版本管理

7.3 服务层 (Agent 的身体)

组件	选型	AI-Native 角色
Web 框架	**FastAPI** (Python)	Agent 与外界的通讯接口（接收钉钉回调、暴露 API、SSE 流式输出）
定时调度	**APScheduler** 或 **Celery Beat**	主动型 Agent 的时钟：触发巡检、晨报、定时分析
异步任务	**Celery** + Redis Broker	重计算任务异步执行：批量评分、报告生成、数据同步

7.4 触点层

组件	选型	AI-Native 角色
主界面	**钉钉** (机器人 + 互动卡片 + 审批)	Agent 与人对话的主通道，零部署成本
辅助界面	**Vue 3** + TypeScript + Element Plus	轻量 Web 看板：数据可视化、管理配置、AI 降级时的操作入口
图表	**ECharts**	漏斗图、趋势图、驾驶舱
构建	**Vite**	前端打包

7.5 钉钉集成

能力	实现方式	Agent 如何使用
SSO 单点登录	钉钉 OAuth2 / JSAPI 免登	识别用户身份，加载角色权限
组织架构同步	通讯录 API + 事件订阅	Agent 了解组织结构，用于分配和权限判断
消息推送	工作通知 API	Agent 主动推送洞察、预警、建议
机器人交互	企业内部机器人 + Webhook	Agent 接收用户消息，返回响应
互动卡片	互动卡片 SDK	Agent 推送可操作卡片（确认/审批/选择）
审批	OA 审批 API	Agent 发起/跟踪审批流程

7.6 DevOps

组件	选型
容器化	**Docker** + Docker Compose
监控	**Prometheus** + **Grafana** (含 Agent 性能指标)
日志	**Loguru** (结构化 JSON) → 日志文件 → 归集

组件	选型
CI/CD	Git + 手动部署 (一期)；后续可接 GitHub Actions

7.7 关键技术决策说明

决策	选择	理由
Agent 框架自研 vs LangChain	**自研**	LangChain 抽象层过厚，Tool-Calling 已是 LLM 原生能力，自研更可控、轻量、易调试
向量库独立 vs pgvector	**pgvector**	一期数据量不大，减少组件数量；向量和业务数据同库便于联合查询
前端重 vs 轻	**轻量看板**	主交互走钉钉，Web 只做可视化和管理，不需要复杂前端
EspoCRM 继续用 vs 自建数据层	**自建**	AI-Native 架构下 Agent 直接读写 PostgreSQL，无需 EspoCRM 中间层
Celery vs 纯 asyncio	**Celery 做重任务**	批量评分、报告生成等需要独立 Worker；轻量异步用 FastAPI 原生 asyncio

7.8 Agent 模型部署分类（本地 vs 云端 API）

> 不是所有 Agent 都必须跑在本地 2×4090 上。
> 核心原则：**涉及内部经营数据的 Agent 必须本地推理；不涉及机密或可完全脱敏的 Agent 可调用云端大模型 API**。
> 使用云端 API 的好处：减轻本地 GPU 负载、获得更强模型能力（云端可用 72B/236B 级别模型）、降低并发瓶颈。

第一类：必须本地部署（涉及内部机密数据）

这些 Agent 的 Prompt 输入中包含客户名称、交易金额、合同条款、经营指标等内部数据，**绝对不能发送到外部服务器**：

Agent	涉及的敏感数据	不能上云的原因
Supervisor Agent	所有用户消息、全部业务数据	作为总调度，它看到一切，是数据汇聚点
销售助手 Agent	客户信息、联系方式、商机金额、报价	直接操作全部 CRM 数据，Prompt 中包含具体客户和交易细节
数据问答 Agent	收入、目标达成率、团队业绩、经营指标	查询和分析的都是核心经营数据
合同审查 Agent	合同条款、金额、付款条件、违约责任	合同是高度机密的商业文件
巡检 Agent	全量客户/商机/合同/回款数据	每日全量扫描，看到所有业务数据
预警 Agent	回款逾期金额、客户流失风险、目标缺口	涉及财务敏感阈值和客户风险评估
催收 Agent	应收款项、回款历史、客户信用状况	财务核心数据，泄露后果严重
晨报 Agent	每日全量经营汇总、团队销售排名	经营日报等于公司经营体检报告
客户洞察 Agent	客户画像、交易历史、决策链、竞对分析	包含竞争情报和客户深度分析
商机预测 Agent	商机金额、赢率、竞争对手、输赢原因	销售管线数据是核心商业机密
漏斗分析 Agent	管线金额分布、转化率、收入预测	这就是公司的营收预测，绝对机密
规划 Agent	销售工作量、客户优先级排序、分配策略	包含客户战略优先级和资源分配决策

> **12 个 Agent 必须本地**，全部运行在 AI 算力节点 (172.16.11.194) 的 vLLM 上。

第二类：可使用云端大模型 API（不涉及机密 / 可脱敏设计）

这些 Agent 的输入**可以设计为完全脱敏**，只传递特征类别而不传递具体客户/金额信息：

Agent	脱敏设计	传给云端的数据示例	绝不传的数据
线索评分 Agent	只传行业、规模区间、来源渠道、地区等类别特征	`{行业: "教育", 规模: "100-500人", 来源: "展会", 地区: "华东", 有明确需求: true}`	公司名称、联系人、电话、邮箱
行为分析 Agent	只传统计特征（匿名化的行为数据）	`{平均跟进周期: 3.2天, 周活动数: 12, 邮件响应率: 68%, 成交率: 23%}`	销售姓名、客户名称、具体金额

> **2 个 Agent 可用云端 API**。推荐使用阿里云 Qwen-Plus API 或 DeepSeek API，享受更强模型能力。

第三类：混合模式（主逻辑本地，子任务可调云端）

任何 Agent 在执行过程中，以下**子任务**不涉及内部数据，可以调用云端 API 获得更好的效果：

可云端执行的子任务	说明	调用时机
行业知识查询	"教育行业机械臂市场规模和趋势？"	客户洞察 Agent 补充行业背景
通用话术生成	"写一段给教育行业客户的开场白"	销售助手 Agent 推荐话术（只传行业，不传客户名）
公开数据解读	解读天眼查返回的工商信息	线索进入时自动分析（天眼查数据本身已公开）
文案润色	把 Agent 生成的日报文字润色得更通顺	晨报 Agent 后处理（内容已完成，只做语言优化）
合同模板起草	"生成一份机械臂采购合同通用模板"	合同审查 Agent 需要风格参考（只传品类，不传具体条款和金额）
竞品公开分析	"XX公司的公开产品线和市场定位"	竞品分析子任务（分析的是公开信息）

架构实现：统一调度 + 双通道推理

```
Agent 推理请求
      |
      ▼
┌──────────────┐
|  模型路由器   |  ← 根据 Agent 类型 + 任务类型 自动选择通道
|  (Model      |
|   Router)    |
└──────┬───────┘
   |       |
   ▼       ▼
┌──────┐ ┌──────────────┐
| 本地  | | 云端 API      |
| vLLM | | (Qwen-Plus/  |
|:8000 | |  DeepSeek)   |
└──────┘ └──────────────┘

# 路由规则（伪代码）
def route_model(agent_name, task_type):
    # 第一类：必须本地
    if agent_name in LOCAL_ONLY_AGENTS:
        return "local_vllm"

    # 第二类：可用云端
    if agent_name in CLOUD_OK_AGENTS:
        return "cloud_api"

    # 第三类：按子任务判断
    if task_type in ["industry_knowledge", "template_generation",
                     "public_data_analysis", "text_polishing"]:
        return "cloud_api"

    # 默认：本地
    return "local_vllm"
```

成本与性能收益

维度	全部本地	本地 + 云端混合
本地 GPU 负载	100% Agent 请求	~80% Agent 请求（核心机密）
云端 API 成本	¥0	约 ¥200-500/月（评分+分析+子任务）
并发能力	受限于 2×4090	核心 Agent 本地 + 辅助任务云端分流
模型能力	30B-A3B（激活 3B）	核心 30B-A3B + 云端可用 72B/236B 级别
延迟	全部低延迟	核心低延迟 + 云端子任务 200-500ms

> **推荐采用混合模式**：12 个核心 Agent 本地推理保证数据安全，2 个可脱敏 Agent + 子任务调用云端 API 缓解 GPU 压力。

8. Agent 记忆模型（数据架构）

> AI-Native 架构下，数据库不是「业务系统的存储」，而是「Agent 的长期记忆」。
> Agent 通过工具读写记忆，就像人类通过大脑存储和检索信息。

8.1 记忆分层与实体关系

```
┌──────────────────────── Agent 结构化记忆 (PostgreSQL) ────────────────────────┐
│                                                                                │
│  ╔═══════════════════ 核心业务实体 (Agent 操作的对象) ═══════════════════╗      │
│  ‖                                                                      ‖      │
│  ‖  ┌──────────┐      ┌──────────┐      ┌──────────┐                   ‖      │
│  ‖  │  用户    │──────<│  线索    │──────>│  客户    │                   ‖      │
│  ‖  │  (User)  │      │  (Lead)  │转化  │(Customer)│                   ‖      │
│  ‖  └──────────┘      └──────────┘      └──────┬───┘                   ‖      │
│  ‖       │                                     │                        ‖      │
│  ‖       │ 负责                            ┌───┴───┐                    ‖      │
│  ‖       │                                 │       │                    ‖      │
│  ‖       ▼                                 ▼       ▼                    ‖      │
│  ‖  ┌──────────┐                  ┌──────────┐ ┌──────────┐            ‖      │
│  ‖  │  商机    │<─────────────────│ 联系人   │ │ 活动记录 │            ‖      │
│  ‖  │(Opportunity)                │(Contact) │ │(Activity)│            ‖      │
│  ‖  └──────┬───┘                  └──────────┘ └──────────┘            ‖      │
│  ‖        │                                                            ‖      │
│  ‖        │ 成交                                                       ‖      │
│  ‖        ▼                                                            ‖      │
│  ‖  ┌──────────┐      ┌──────────┐      ┌──────────┐                   ‖      │
│  ‖  │  合同    │──────>│  订单    │──────>│  回款    │                   ‖      │
│  ‖  │(Contract)│      │ (Order)  │      │(Payment) │                   ‖      │
│  ‖  └──────────┘      └──────────┘      └──────────┘                   ‖      │
│  ‖                                                                      ‖      │
│  ‖  横切实体:                                                           ‖      │
│  ‖  ┌────────┐ ┌────────┐ ┌────────┐ ┌────────┐ ┌────────┐            ‖      │
│  ‖  │ 产品   │ │ 任务   │ │ 工单   │ │ 目标   │ │ 审批   │            ‖      │
│  ‖  │Product │ │ Task   │ │Ticket  │ │Target  │ │Approval│            ‖      │
│  ‖  └────────┘ └────────┘ └────────┘ └────────┘ └────────┘            ‖      │
│  ╚═══════════════════════════════════════════════════════════════════╝      │
│                                                                                │
│  ╔═══════════════ Agent 专属记忆 (Agent 自身的数据) ═══════════════╗         │
│  ‖                                                                      ‖      │
│  ‖  ┌──────────────┐  ┌──────────────┐  ┌──────────────────┐          ‖      │
│  ‖  │ 审计事实流   │  │ Agent操作日志 │  │ AI 生成内容缓存  │          ‖      │
│  ‖  │(AuditEvent)  │  │ (AgentLog)   │  │ (AICache)        │          ‖      │
│  ‖  │              │  │              │  │                  │          ‖      │
│  ‖  │· 不可变事件流 │  │· agent_id    │  │· 客户画像摘要    │          ‖      │
│  ‖  │· who/what/when│  │· 推理链路    │  │· 线索评分结果    │          ‖      │
│  ‖  │· 操作前后快照 │  │· 工具调用记录│  │· 商机赢率预测    │          ‖      │
│  ‖  │· 支持全链路回放│  │· 耗时/token  │  │· 各类分析报告    │          ‖      │
│  ‖  └──────────────┘  └──────────────┘  └──────────────────┘          ‖      │
│  ‖                                                                      ‖      │
│  ‖  ┌──────────────┐  ┌──────────────┐                                ‖      │
│  ‖  │ 知识向量表   │  │ 反馈学习表   │                                ‖      │
│  ‖  │(Knowledge)   │  │ (Feedback)   │                                ‖      │
│  ‖  │(pgvector)    │  │              │                                ‖      │
│  ‖  │              │  │· 推荐采纳/拒绝│                               ‖      │
│  ‖  │· 赢单/丢单经验│  │· 用户评价    │                               ‖      │
│  ‖  │· 产品知识文档 │  │· 预测vs实际  │                               ‖      │
│  ‖  │· 行业特征知识 │  │· 用于模型优化│                               ‖      │
│  ‖  │· 话术模板    │  │              │                                ‖      │
│  ‖  └──────────────┘  └──────────────┘                                ‖      │
│  ╚═══════════════════════════════════════════════════════════════════╝      │
└────────────────────────────────────────────────────────────────────────────┘
```

8.2 核心实体说明

> 每个实体都包含 `ai_` 前缀的 Agent 专属字段，这是 AI-Native 的关键设计。

实体	说明	标准字段	Agent 专属字段
User	用户 (钉钉同步)	dd_user_id, 姓名, 部门, 角色	ai_behavior_profile (行为画像)
Lead	线索	来源, 联系方式, 状态, 负责人	ai_score (评分), ai_score_reason (依据), ai_suggested_assignee
Customer	客户	名称, 行业, 规模, 分级, 负责人	ai_portrait (画像摘要), ai_health_score, ai_churn_risk
Contact	联系人	姓名, 职位, 电话, 所属客户	ai_decision_role (决策链角色), ai_relationship_strength
Opportunity	商机	阶段, 金额, 预计成交日, 客户	ai_win_rate, ai_stall_days, ai_next_action, ai_risk_factors
Activity	活动 (拜访/电话/邮件)	类型, 内容, 时间, 关联实体	ai_summary (AI摘要), ai_extracted_todos
Contract	合同	编号, 金额, 期限, 状态	ai_risk_review (条款审查结果), ai_renewal_suggestion
Order	订单	产品, 数量, 金额, 状态	ai_delivery_risk, ai_anomaly_flag
Payment	回款	金额, 计划日, 实际日, 状态	ai_overdue_risk, ai_collection_strategy
Target	目标	类型, 金额, 周期, 负责人	ai_completion_prediction, ai_gap_analysis
Task	任务	标题, 状态, 截止日, 关联实体	ai_generated (是否AI生成), ai_priority_reason
Product	产品	名称, 编码, 价格, 状态	ai_cross_sell_tags (交叉销售标签)

8.3 Agent 专属表设计

AuditEvent (审计事实流)

```
不可变的事件溯源表，记录所有业务操作的事实。
Agent 的「长期陈述性记忆」—发生过什么。

字段:
  id             BIGSERIAL PRIMARY KEY
  event_id       UUID UNIQUE           -- 事件唯一ID
  timestamp      TIMESTAMPTZ NOT NULL  -- 精确时间
  actor_type     VARCHAR(20)           -- 'user' | 'agent' | 'system'
  actor_id       VARCHAR(100)          -- 操作者ID
  agent_id       VARCHAR(50)           -- 如果是Agent操作，哪个Agent
  entity_type    VARCHAR(50)           -- 'lead' | 'customer' | 'opportunity' ...
  entity_id      UUID                  -- 实体ID
  action         VARCHAR(50)           -- 'create' | 'update' | 'delete' | 'stage_change' ...
  changes        JSONB                 -- 变更前后对比 {"field": {"old": x, "new": y}}
  context        JSONB                 -- 操作上下文（来源、推理链路等）

索引: timestamp, entity_type+entity_id, actor_id
分区: 按月分区
策略: INSERT ONLY，不允许 UPDATE/DELETE
```

AgentLog (Agent 操作日志)

```
记录 Agent 每次推理的完整链路。
用于调试、优化和审计 Agent 行为。

字段:
  id             BIGSERIAL PRIMARY KEY
  trace_id       UUID UNIQUE           -- 追踪ID (一次完整交互)
  agent_type     VARCHAR(50)           -- 'supervisor' | 'lead_score' | 'insight' ...
  trigger_type   VARCHAR(30)           -- 'chat' | 'event' | 'schedule' | 'agent_delegate'
  trigger_detail JSONB                 -- 触发详情
  user_id        VARCHAR(100)          -- 关联用户 (如有)
  reasoning      JSONB                 -- 推理链路 [{step, thought, action, result}]
  tools_called   JSONB                 -- 工具调用记录 [{tool, params, result, duration_ms}]
  llm_calls      JSONB                 -- LLM调用 [{prompt_tokens, completion_tokens, duration_ms}]
  final_output   TEXT                  -- 最终输出
  total_duration INTEGER               -- 总耗时(ms)
  status         VARCHAR(20)           -- 'success' | 'partial' | 'error' | 'timeout'
  created_at     TIMESTAMPTZ
```

Knowledge (知识向量表)

```
Agent 的语义记忆, 使用 pgvector 存储向量。

字段:
  id             BIGSERIAL PRIMARY KEY
  category       VARCHAR(50)           -- 'win_case' | 'lose_case' | 'product' | 'industry' | 'faq'
  title          VARCHAR(500)          -- 知识标题
  content        TEXT                  -- 原始文本
  embedding      vector(1024)          -- 向量表示 (一期: 阿里云百炼 text-embedding-v3 / 二期: 本地 BGE-M3, 均为 1024 维)
  metadata       JSONB                 -- 元数据 (来源、时间、关联实体等)
  created_at     TIMESTAMPTZ
  updated_at     TIMESTAMPTZ

索引: embedding 使用 HNSW 索引
```

9. 接口设计概要

> AI-Native 架构下，API 分两类：
> **1) Agent 对外接口** — 人/钉钉与 Agent 的交互入口
> **2) Agent 工具接口** — Agent 内部调用的工具函数（不直接暴露给用户）

9.1 对外 API：Agent 交互入口

核心理念：用户不需要知道有哪些 API，只需要和 Agent 对话

接口	方法	说明
`/api/v1/agent/chat`	POST (SSE)	**主入口**：自然语言对话，流式返回。Agent 理解意图后自动执行所有操作
`/api/v1/agent/chat/{session_id}`	GET	获取对话历史
`/api/v1/agent/confirm/{action_id}`	POST	确认/拒绝 Agent 提出的高风险操作
`/api/v1/dingtalk/callback`	POST	钉钉机器人回调入口
`/api/v1/dingtalk/event`	POST	钉钉事件订阅回调

接口	方法	说明
`/api/v1/dingtalk/card/action`	POST	钉钉互动卡片操作回调
`/api/v1/auth/dingtalk`	POST	钉钉 SSO 登录，返回 JWT
`/api/v1/auth/refresh`	POST	Token 刷新

示例：一个 API 搞定一切

```
传统 CRM API (用户需要了解几十个接口):
  POST /api/v1/leads/           -- 创建线索
  GET  /api/v1/leads/           -- 查询线索
  PUT  /api/v1/leads/{id}       -- 更新线索
  GET  /api/v1/customers/       -- 查询客户
  POST /api/v1/opportunities/   -- 创建商机
  PUT  /api/v1/opportunities/{id}/stage  -- 推进阶段
  GET  /api/v1/analytics/funnel -- 漏斗报表
  ...(几十个接口)

AI-Native API (用户只需要一个入口):
  POST /api/v1/agent/chat
  {
    "message": "帮我查一下华东区这个月新增了多少线索",
    "session_id": "xxx"
  }

  Agent 内部自动: 理解意图 → 调用查询工具 → 聚合数据 → 返回自然语言答案
```

9.2 看板 API：Web 可视化 & 降级操作

> 轻量看板的数据 API，仅用于可视化展示和 AI 降级时的手动操作。

分组	路径前缀	说明
仪表盘	`/api/v1/dashboard/`	首页看板数据（今日概览、漏斗、目标进度）
线索	`/api/v1/leads/`	线索列表/详情/CRUD (降级用)
客户	`/api/v1/customers/`	客户列表/详情/360视图
商机	`/api/v1/opportunities/`	商机列表/详情/漏斗视图
活动	`/api/v1/activities/`	活动时间线
分析	`/api/v1/analytics/`	报表/图表数据
管理	`/api/v1/admin/`	Agent 配置/日志/健康状态

9.3 内部 API：Agent 与 Agent 之间

接口	说明
`POST /internal/agent/delegate`	Supervisor 委派任务给子 Agent
`POST /internal/agent/event`	事件总线推送
`POST /internal/llm/chat`	统一 LLM 调用入口 (vLLM 代理)
`POST /internal/llm/embedding`	Embedding 生成

9.4 统一响应格式

```
// Agent 对话响应 (SSE 流式)
event: message
data: {"type": "thinking", "content": "正在查询华东区线索数据..."}

event: message
data: {"type": "tool_call", "tool": "query_leads", "status": "executing"}

event: message
data: {"type": "answer", "content": "华东区本月新增线索 47 条，同比增长 23%..."}

event: message
data: {"type": "action_card", "card": {...}}  // 钉钉互动卡片数据

event: done
data: {"trace_id": "xxx", "tokens_used": 1234, "duration_ms": 2300}


// 看板 API 响应 (标准 RESTful)
{
  "success": true,
  "code": 200,
  "message": "ok",
  "data": {
    "items": [...],
    "total": 47,
    "page": 1,
    "page_size": 20
  }
}
```

9.5 认证方式

入口	认证方式
钉钉机器人	钉钉签名验证 (Webhook Secret)
钉钉互动卡片	钉钉回调 Token 验证
Web 看板	JWT Bearer Token (钉钉 SSO 获取)
内部 Agent 通信	内网 IP 白名单 + 内部 Token

10. 安全设计与 Agent 风控

> AI-Native 架构的安全核心不仅是「防外部攻击」，更是「控制 Agent 行为」。

10.1 基础安全

层面	措施
认证	钉钉 SSO + JWT，Token 有效期 30 分钟，Refresh Token 7 天
授权	RBAC (管理员/经理/销售)，数据权限按部门隔离
传输	内网通信；对外接口 HTTPS (Nginx SSL 终结)

层面	措施
数据	敏感字段加密存储；SQL 参数化查询；API Key 脱敏
审计	所有操作记录到 AuditEvent 事实流，不可篡改

10.2 Agent 行为管控 (三级风险控制)

风险等级	操作类型	Agent 行为	举例
低风险	查询/读取	自动执行，直接返回	查询线索列表、查看客户详情
中风险	单条数据修改	执行 + 事后通知	更新商机阶段、记录活动
高风险	批量/财务/删除	暂停 + 推送确认卡片 + 等待人工审批	批量分配线索、删除客户、修改合同金额

10.3 Agent 安全规则

```
1. 所有 Agent 生成内容必须标记: agent_id + model_version + timestamp
2. Agent 不能绕过 RBAC 权限 —— 工具调用前检查当前用户权限
3. 写操作必须幂等 —— 相同操作重复执行不产生副作用
4. 所有写操作可回滚 —— AuditEvent 记录变更前后快照
5. Agent 推理链路完整记录 —— 可审计、可回放、可追责
6. LLM Prompt 注入防护 —— 输入清洗 + Prompt 边界隔离
7. 敏感数据不进 Prompt —— 客户密码/银行卡号等不传入 LLM
```

10.4 AI 降级保障

场景	降级行为
LLM 服务不可用	AI 字段显示"AI 暂不可用"，Web 看板 CRUD 正常使用
LLM 响应超时 (>30s)	返回缓存结果 (如有) 或提示稍后重试
Agent 引擎崩溃	自动重启 + 钉钉通知管理员 + 纯 CRUD 模式运行
AI 结果置信度过低	标记"低置信度"，不执行仅供参考
AI 连续异常	自动熔断，停止该类 Agent 调用，通知管理员

11. 分期策略

> 100+ 功能点需分期交付。AI-Native 架构的分期原则：**先让 Agent 能对话，再让 Agent 越来越聪明**。
> 开发模式：5 台机器并行开发 + Copilot Agent 辅助（详见 §13），orchestrator 本地自动测试评估（详见 05-测试方案）。
> 模型策略：核心 Agent 本地 vLLM，可脱敏 Agent + 子任务走云端 API（详见 §7.8）。

一期：Agent 对话 + 核心 CRM（8 周）

目标：Agent 能通过钉钉完成基础 CRM 操作 + 关键 AI 洞察

Sprint 1-2（第 1-2 周）：基础设施 + 接口契约

机器	交付内容
4090 (AI 算力)	vLLM 部署调优、Agent 框架骨架、Supervisor Agent v0.1、模型路由器（本地/云端双通道）
mac_min_8T (CRM)	PostgreSQL + pgvector + Redis 搭建、数据库 Schema v1（线索/客户/商机/活动表）、FastAPI 项目骨架

机器	交付内容
gateway (网关)	Nginx 反向代理配置、钉钉应用注册 + 回调接收
data_center (数据中心)	Git 裸仓库搭建、开发环境自动化脚本、基础监控
共享产出	`contracts/` 接口契约 v1（agent-api.yaml + crm-api.yaml + agent-tools.yaml）

> Sprint 1-2 结束标准：5 台机器能跑通 "发消息 → Agent 收到 → 调用 vLLM → 返回文字" 端到端链路，orchestrator 自动评估正常运行。

Sprint 3-4（第 3-4 周）：销售助手 Agent + 线索/客户

机器	交付内容
4090	销售助手 Agent（自然语言 CRUD）、线索评分 Agent（走云端 API，脱敏设计）、CRM Tool 5 个（create/query/update_lead, create/query_customer）
mac_min_8T	线索 REST API + 客户 REST API、线索池分配逻辑、钉钉 SSO + JWT 认证、种子数据脚本
gateway	钉钉机器人对话闭环、SSO 单点登录联调
data_center	审计日志表 + 写入逻辑、PG 自动备份脚本

> Sprint 3-4 结束标准：销售在钉钉说"帮我创建一个线索，XX公司张总13800138000"，Agent 完成创建并返回确认。

Sprint 5-6（第 5-6 周）：商机/活动 + AI 洞察

机器	交付内容
4090	商机预测 Agent（本地）、客户洞察 Agent（本地）、CRM Tool 6 个（商机 CRUD + 活动 CRUD）、行业知识子任务走云端 API
mac_min_8T	商机 REST API + 活动 REST API、阶段流转逻辑、漏斗统计接口
gateway	钉钉互动卡片（线索领取、商机推进确认）
data_center	Celery Worker 搭建、异步评分任务

> Sprint 5-6 结束标准：商机阶段推进时自动计算赢率并推送钉钉卡片。

Sprint 7-8（第 7-8 周）：Web 看板 + 联调 + 上线

机器	交付内容
4090	Agent 全量联调、Prompt 调优、行为分析 Agent（云端 API）、降级逻辑
mac_min_8T	Vue3 Web 看板（仪表盘 + 列表 + 详情 + 漏斗图）、API 性能优化
gateway	完整 Nginx 配置（SSL + 限流 + 路由）、全链路联调
data_center	Prometheus + Grafana 监控面板、PG 流复制、日志归集
共享产出	从开发模式切换到运行模式（§12.7）

> Sprint 7-8 结束标准：一期所有功能端到端可用，系统切换到运行模式。

一期交付清单

类别	数量	明细
Agent	6 个	Supervisor、销售助手、线索评分 ☁️ 、客户洞察、商机预测、行为分析 ☁️
CRM Tool	11 个	线索×3 + 客户×3 + 商机×3 + 活动×2

类别	数量	明细
REST API	4 组	线索/客户/商机/活动 CRUD + 统计
前端页面	6 个	仪表盘 + 线索/客户/商机/活动列表 + 详情页
钉钉能力	4 项	机器人对话、互动卡片、SSO、组织同步
数据库表	~12 张	业务表 + Agent 日志 + 审计 + 用户

> ☁️ = 使用云端大模型 API（脱敏后调用，详见 §7.8）

二期：Agent 主动智能 + 业务闭环（8 周）

目标：Agent 从「被动响应」进化为「主动服务」；跑通合同→订单→回款闭环

Sprint 9-10（第 9-10 周）：主动型 Agent

机器	交付内容
4090	巡检 Agent（本地）、预警 Agent（本地）、晨报 Agent（本地，文案润色子任务走云端）、规划 Agent（本地）
mac_min_8T	事件总线（Redis Stream）、目标管理 API、日报/周报 API
gateway	钉钉消息推送优化（分级通知策略）、审批流集成
data_center	Celery Beat 定时任务配置（巡检/晨报/规划的 cron 调度）

Sprint 11-12（第 11-12 周）：合同/订单/回款

机器	交付内容
4090	合同审查 Agent（本地）、催收 Agent（本地）、合同模板生成子任务走云端
mac_min_8T	合同/订单/回款 REST API、状态机逻辑
gateway	钉钉审批流闭环（合同审批 → Agent 自动处理结果）
data_center	回款到期监控任务、异常检测定时任务

Sprint 13-14（第 13-14 周）：数据问答 + 知识库

机器	交付内容
4090	数据问答 Agent（本地）、漏斗分析 Agent（本地）、知识库向量索引
mac_min_8T	报表统计 API、知识库管理 API（上传/索引/检索）
gateway	钉钉互动卡片（每日简报、审批操作、报表快链）
data_center	知识库文档导入脚本（产品资料/话术/FAQ）

Sprint 15-16（第 15-16 周）：联调 + 稳定化

机器	交付内容
all	全量联调、性能优化、Prompt 调优、文档更新、bug 修复、压力测试

二期交付清单

类别	数量	明细
Agent	+7 个	巡检、预警、晨报、规划、合同审查、催收、数据问答、漏斗分析（全部本地，子任务走云端）
CRM Tool	+10 个	合同×3 + 订单×2 + 回款×2 + 目标×2 + 知识检索×1
REST API	+5 组	合同/订单/回款/目标/知识库
前端页面	+5 个	合同/订单/回款列表 + 目标看板 + 知识库管理
钉钉能力	+3 项	审批流闭环、互动卡片操作、项目群自动创建

三期：多 Agent 协作 + 跨系统智能（8 周）

目标：多 Agent 协同编排；打通 ERP/MES/分贝通

Sprint	交付内容
17-18	多 Agent 编排框架（Agent 间委派/并行/汇总）、ERP U9 接口对接
19-20	MES/分贝通/物流集成、Agent 反馈闭环
21-22	高级分析（收入预测、CLV、产品趋势）、智能风控
23-24	语音交互、Agent 学习与进化、全量联调与上线

整体里程碑

```
Week 1   2   3   4   5   6   7   8   |   9  10  11  12  13  14  15  16  |  17  ...  24
————————————————————————————————————|————————————————————————————————|————————————
       一期: Agent 对话 + 核心 CRM   |    二期: 主动智能 + 业务闭环       | 三期: 协作+集成
                                     |                                  |
[接口契约] [助手+线索] [商机+AI] [看板]| [主动Agent] [合同回款] [问答] [稳定]| [编排] ... [上线]
    S1-2      S3-4      S5-6    S7-8 |   S9-10     S11-12   S13-14 S15-16|  S17-18 ... S23-24
                                     |                                  |
里程碑:   ▲ 能对话  ▲ 能做CRM  ▲ 一期上线|  ▲ 主动推送  ▲ 闭环  ▲ 二期上线    |         ▲ 三期上线
```

12. 开发阶段部署方案

> **现实约束：4 台硬件既是开发环境，也是最终运行环境，没有独立的"生产集群"。**
> 因此部署策略不是"开发机 → 上线到生产机"，而是**同一组机器在不同阶段切换运行模式**。

12.1 同一硬件的两种运行模式

内网 4 台机器始终承担 §6 所定义的角色（网关/CRM/AI/数据中心），只是运行方式随项目阶段演进；第 5 台机器 (orchestrator) 承担调度 + Review + 测试 + 预发布验收：

维度	开发模式（一期开发阶段）	运行模式（交付上线后）
部署方式	直接运行 / Poetry venv，热重载优先	Docker Compose 编排，systemd 托管
Nginx	可选，联调时启用	完整反向代理 + SSL + 限流
PostgreSQL	单实例，seed 脚本灌模拟数据	主从流复制，定期备份
Redis	单实例，默认配置	持久化 + AOF，密码保护

维度	开发模式（一期开发阶段）	运行模式（交付上线后）
vLLM	相同参数，常驻运行	相同参数，常驻运行
监控	可选，按需启用	Prometheus + Grafana 常驻
前端	Vite dev server + HMR	Nginx 托管构建后的静态文件
Agent 引擎	uvicorn --reload 热重载	gunicorn/uvicorn workers + systemd
数据	开发测试数据，可随时重置	真实业务数据，不可丢失

> **关键认知**：vLLM 在两种模式下配置完全一致——模型加载慢、显存宝贵，没必要区分。
> 开发阶段的 vLLM 就是上线后的 vLLM，**从第一天起就是生产级运行**。

12.2 各节点角色（开发 + 运行共存）

每台机器的**物理角色固定**（和 §6 一致），开发阶段只是把服务改为热重载启动：

```
┌──────────────────────────────────────────────────────────────────────────┐
│          内网 4 台机器 = 开发环境 = 运行环境 + 云端 1 台 = 测试环境        │
│                                                                          │
│  ┌── AI 算力节点 (172.16.11.194) ──────────────────────────────┐        │
│  │ 永久角色: AI 推理 + Agent 引擎                               │        │
│  │                                                              │        │
│  │  [常驻] vLLM 推理服务 (开发/运行 配置一致)                   │        │
│  │  $ vllm serve models/Qwen3-30B-A3B \                         │        │
│  │      --tensor-parallel-size 2 \                              │        │
│  │      --enable-prefix-caching \                               │        │
│  │      --gpu-memory-utilization 0.85 \                         │        │
│  │      --port 8000                                             │        │
│  │                                                              │        │
│  │  [开发时] Agent 引擎 → uvicorn --reload :8100                │        │
│  │  [运行时] Agent 引擎 → Docker/systemd :8100                  │        │
│  │                                                              │        │
│  │  + VS Code Remote SSH 开发 Agent/Prompt/Tool 代码            │        │
│  └──────────────────────────────────────────────────────────────┘        │
│                                                                          │
│  ┌── 边缘 CRM 设备 (172.16.12.50) ──────────────────────────┐        │
│  │ 永久角色: CRM 后端 + 前端 + PostgreSQL + Redis               │        │
│  │                                                              │        │
│  │  [常驻] PostgreSQL 16                                        │        │
│  │  $ brew services start postgresql@16                         │        │
│  │                                                              │        │
│  │  [常驻] Redis 7                                              │        │
│  │  $ brew services start redis                                 │        │
│  │                                                              │        │
│  │  [开发时] CRM 后端 → uvicorn --reload :8900                  │        │
│  │  [运行时] CRM 后端 → Docker/gunicorn :8900                   │        │
│  │                                                              │        │
│  │  [开发时] 前端 → Vite dev server + HMR :3000                 │        │
│  │  [运行时] 前端 → npm run build → Nginx 托管静态文件          │        │
│  │                                                              │        │
│  │  + VS Code 本地开发后端 API / 前端 / DB 迁移                 │        │
│  └──────────────────────────────────────────────────────────────┘        │
│                                                                          │
│  ┌── 网关服务器 (172.16.14.215) ──────────────────────────────┐        │
│  │ 永久角色: 流量入口 + 钉钉回调                                │        │
│  │                                                              │        │
│  │  [开发时] Nginx 可选启动 (联调/钉钉对接时开)                 │        │
│  │  [运行时] Nginx 常驻 + SSL + 限流 + 钉钉回调                 │        │
│  │                                                              │        │
│  │  + VS Code Remote SSH 开发 Nginx 配置和集成代码              │        │
│  └──────────────────────────────────────────────────────────────┘        │
│                                                                          │
│  ┌── 数据中心 (172.16.14.90) ──────────────────────────────┐        │
│  │ 永久角色: 数据备份 + 后台任务 + 监控                         │        │
│  │                                                              │        │
│  │  [开发时] PG 备库/Celery/监控 按需启用                       │        │
│  │  [运行时] PG 流复制 + Celery Worker + Prometheus 常驻        │        │
│  │                                                              │        │
│  │  + VS Code Remote SSH 开发运维脚本和监控配置                 │        │
│  └──────────────────────────────────────────────────────────────┘        │
│                                                                          │
└──────────────────────────────────────────────────────────────────────────┘
```

12.3 启动顺序（开发模式）

分 3 层依次启动，每层服务就绪后再启动下一层：

```
第 1 层：基础设施 — 启动一次，永久常驻（开发和运行都一样）
──────────────────────────────────────────
[CRM 设备]  PostgreSQL  → 创建 ai_crm 数据库 + 执行 migrations
[CRM 设备]  Redis       → 默认配置启动即可
[AI 节点]   vLLM        → 加载模型约 2-3 分钟，启动后常驻（tmux/screen）

  ★ 这 3 个服务无论开发还是运行，配置和启动方式基本一致。
  ★ vLLM 尤其不要停，它就是生产推理服务，从第一天起常驻。

第 2 层：业务服务 — 开发时热重载，运行时 Docker/systemd
──────────────────────────────────────────
[CRM 设备]  CRM 后端     → uvicorn --reload :8900    （开发）
                           → docker compose up crm     （运行）
[AI 节点]   Agent 引擎   → uvicorn --reload :8100    （开发）
                           → docker compose up agent    （运行）

第 3 层：辅助服务 — 按需启动
──────────────────────────────────────────
[CRM 设备]  前端 dev     → Vite HMR :3000             （开发）
                           → nginx serve dist/          （运行）
[网关]      Nginx        → 联调/上线时启动
[数据中心]  Celery       → 开发异步任务时启动
[数据中心]  PG 备库      → 上线前配置流复制
[数据中心]  监控         → 上线前配置 Prometheus + Grafana
```

12.4 关键开发配置

环境变量配置 (.env.dev)

```
# 数据库（CRM 设备 IP）
DATABASE_URL=postgresql+asyncpg://dev:dev@172.16.12.50:5432/ai_crm
REDIS_URL=redis://172.16.12.50:6379/0

# vLLM（AI 算力节点 IP）
VLLM_BASE_URL=http://172.16.11.194:8000/v1
AGENT_ENGINE_URL=http://172.16.11.194:8100

# Agent 引擎连接数据库（跨节点）
AGENT_DATABASE_URL=postgresql+asyncpg://dev:dev@172.16.12.50:5432/ai_crm
AGENT_REDIS_URL=redis://172.16.12.50:6379/0

# 开发模式标志
ENV=development
DEBUG=true
LOG_LEVEL=DEBUG
```

跨节点通信矩阵 (开发阶段)

```
CRM 后端（.12.50:8900）  ──HTTP──>  Agent 引擎（.11.194:8100）
Agent 引擎（.11.194:8100）──HTTP──>  vLLM（.11.194:8000）        # 同机，零延迟
Agent 引擎（.11.194:8100）──TCP───>  PostgreSQL（.12.50:5432）   # 跨节点
Agent 引擎（.11.194:8100）──TCP───>  Redis（.12.50:6379）        # 跨节点
前端（.12.50:3000）        ──HTTP──>  CRM 后端（.12.50:8900）      # 同机
```

12.5 开发数据初始化

```
# 1. 创建数据库
createdb -h 172.16.12.50 -U dev ai_crm

# 2. 执行迁移（Alembic）
cd ai-crm && alembic upgrade head

# 3. 灌入种子数据（开发用的模拟数据）
python scripts/seed_dev_data.py

# seed 脚本内容：
# - 10 个测试客户（含不同行业和规模）
# - 20 条测试线索（不同来源和状态）
# - 5 个商机（覆盖各阶段）
# - 3 个钉钉测试用户映射
# - Agent Prompt 模板初始化
```

12.6 验证检查单

开发模式和运行模式使用相同的检查项（端口一致，无需切换）：

#	检查项	验证方法	预期结果
1	vLLM 正常运行	`curl http://172.16.11.194:8000/v1/models`	返回模型列表
2	Agent 能调用 LLM	`curl -X POST http://172.16.11.194:8100/chat -d '{"message":"你好"}'`	返回 Agent 响应
3	CRM 后端启动	`curl http://172.16.12.50:8900/health`	返回 `{"status":"ok"}`
4	数据库连通	`psql -h 172.16.12.50 -U dev -d ai_crm -c 'SELECT 1'`	返回 1
5	Redis 连通	`redis-cli -h 172.16.12.50 ping`	返回 PONG
6	前端可访问	浏览器打开 `http://172.16.12.50:3000`	显示登录页
7	端到端测试	通过前端/钉钉发送「帮我创建一个线索」	Agent 执行创建并返回结果

12.7 从开发模式切换到运行模式

上线交付时，**在同一组机器上原地切换**（不迁移、不换机器）：

```
步骤 1: 代码冻结
────────────────────────────────────────
  git checkout main && git tag v1.0.0
  所有节点 git pull origin main

步骤 2: 构建
────────────────────────────────────────
  [CRM 设备] cd frontend && npm run build     # 构建前端静态文件
  [CRM 设备] docker compose build crm          # 构建后端镜像
  [AI 节点]  docker compose build agent        # 构建 Agent 镜像

步骤 3: 停开发服务 → 启运行服务
────────────────────────────────────────
  [CRM 设备] 停 uvicorn --reload → docker compose up -d crm
  [AI 节点]  停 uvicorn --reload → docker compose up -d agent
  [AI 节点]  vLLM 不动，继续常驻
  [网关]     启动 Nginx 完整配置（SSL + 反向代理）
  [数据中心] 启动 PG 流复制 + Celery + Prometheus + Grafana

步骤 4: 数据迁移
────────────────────────────────────────
  [CRM 设备] alembic upgrade head              # 确保最新表结构
  [CRM 设备] python scripts/init_prod_data.py  # 初始化正式数据（如需要）
  清除开发测试数据 或 保留（按需决定）

步骤 5: 验证
────────────────────────────────────────
  执行 §12.6 全部检查项
  + Nginx SSL 访问验证
  + 钉钉机器人回调验证
  + PG 备库同步状态验证
```

> **核心优势**：开发和运行用同一组机器、同一套端口、同一个代码仓库。
> 切换过程不涉及任何网络拓扑变更，只是把「直接运行」换成「Docker/systemd 托管」。

13. 全自动化开发流水线

> **核心模式：你描述方案 → Orchestrator 自动调度 → 5 台机器协作完成编码→Review→测试→部署，零人工参与。**
> 你的唯一职责是编写/更新 `docs/` 和 `contracts/` 中的设计文档与接口契约。
> aider 已覆盖核心编码循环（上下文读取→代码生成→写入→commit→测试→自动修复→重试）。Orchestrator 是**薄调度层**：解析任务列表 → SSH 分发 aider 指令到 5 台机器 → 收集结果 → 判定下一步。AutoReview/DocParser 走云端 API，Agent 测试推理调用 4090 vLLM A。
> 详细设计见 08-自动化开发流水线.md。

13.1 全自动化架构总览

```
                    ┌──────────────────────────┐
                    │   你（方案设计者）          │
                    │   编写 docs/ + contracts/  │
                    │   git push 更新设计文档     │
                    └──────────────────────────┘
                                │ docs changed
                                ▼
┌──────────────────────────────────────────────────────────────────────┐
│                orchestrator (172.16.14.201)                          │
│                    ┌──────────────────────────┐                      │
│                    │  🎯 Orchestrator（薄调度层）  │                      │
│                    │  ~200 行 shell/Python      │                      │
│                    │ 解析任务 → SSH 分发 aider   │                      │
│                    │ → 收集 exit code → 通知     │                      │
│                    │ (aider 已含编码+测试+修复)  │                      │
│                    └──────────────────────────┘                      │
│                      │    │    │    │                                │
└──────────────────────┼────┼────┼────┼────────────────────────────────┘
          SSH + aider  │    │    │    │  SSH + aider
       ┌───────────────┘    │    │    └──────────────────┐
       ▼                    ▼    ▼                       ▼
┌──────────────┐  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐
│    4090      │  │  mac_min_8T  │  │   gateway    │  │  data_center │
│  (.11.194)   │  │  (.12.50)    │  │  (.14.215)   │  │  (.14.90)    │
│              │  │              │  │              │  │              │
│ aider 编码   │  │ aider 编码   │  │ aider 编码   │  │ aider 编码   │
│ agent/ 目录  │  │ crm/+frontend/│  │ deploy/nginx/│  │ deploy/monitor │
│ + vLLM 推理  │  │ + Git裸仓库  │  │ + dingtalk/  │  │ + scripts/   │
│ (为所有aider │  │ + DB         │  │              │  │ + PG热备+监控 │
│  提供 API)   │  │              │  │              │  │              │
└──────────────┘  └──────────────┘  └──────────────┘  └──────────────┘
       │                │                │                  │
       └────────────────┴────────────────┘                  │
                        │ git push                          │
                        ▼                                   │
                ┌──────────────────┐  post-receive hook    │
                │ Git 裸仓库        │──────────────────────►│
                │ mac_min_8T       │                       │
                └──────────────────┘◄── 测试报告 ──────────│
                        │                                   │
                        ▼                                   │
                Orchestrator 收到报告 → 判定 → 下一批任务          │
```

13.2 五台机器的角色定义

机器	SSH 别名	位置	职责	自动化角色
orchestrator	`ssh orchestrator`	内网 172.16.14.201	**Orchestrator 薄调度层 + Staging**	Orchestrator 调度脚本（基于 aider 已有编码循环，负责跨机器任务编排） + 云端 API (AutoReview/DocParser) + **pytest 本地执行** + 云端API多模型 Judge + **预发布验收**（独立 Staging DB/配置） + aider 编码 `orchestrator/` （~~vLLM B 已~~移除）
4090	`ssh 4090`	内网 172.16.11.194	Agent 引擎 + vLLM A	**vLLM 实例 A (生产 Agent 推理专用)** + aider 自动编码 `agent/`

机器	SSH 别名	位置	职责	自动化角色
mac_min_8T	`ssh mac_min_8T`	内网 172.16.12.50	CRM 后端 + 前端 + DB	aider 自动编码 `crm/` + `frontend/` + Git 裸仓库 + PostgreSQL
gateway	`ssh gateway`	内网 172.16.14.215	网关 + 钉钉	aider 自动编码 `deploy/nginx/` + `deploy/dingtalk/`
data_center	`ssh data_center`	内网 172.16.14.90	数据保障	aider 编码 `deploy/monitoring/` + `scripts/` + PG热备+监控+日志归集

> ~~aliyun-agent（云端 L20, Staging）~~ — 已移除，预发布验收由 orchestrator 承担，vLLM C 取消。

本地 SSH 配置 (~/.ssh/config) — 与 v1.0 相同，Orchestrator 也使用这些别名：

```
Host 4090
    HostName 172.16.11.194
    User user

Host mac_min_8T
    HostName 172.16.12.50
    User edge_sale

Host gateway
    HostName 172.16.14.215
    User realman

Host data_center
    HostName 172.16.14.90
    User realman

Host orchestrator
    HostName 172.16.14.201
    User user
```

13.3 全自动开发循环

人不参与编码循环。Orchestrator 读取设计文档后，驱动以下全自动循环：

```
                    全自动开发循环 (Autonomous Dev Loop)
                    ═══════════════════════════════════

① 解析设计文档 + Sprint 任务卡
   │  Orchestrator 的 Doc Parser 读取 docs/ 和 contracts/
   │  结合 vLLM 将文档拆解为结构化编码任务
   │  按依赖关系排序，入队到 Redis 任务队列
   ▼
② aider 并行编码 (最多 4 台机器同时)
   │  Orchestrator 通过 SSH 在 4 台机器上启动 aider
   │  aider 通过一步 API(OpenAI兼容) 调用 Claude Opus 4.6 生成代码
   │  每个编码任务 5-15 分钟
   │  aider 自动 commit
   ▼
③ 三层自动 Review
   │  Layer 1: 静态检查 (py_compile + ruff)
   │  Layer 2: 契约对齐检查 (LLM 对比 contracts/)
   │  Layer 3: 设计符合度检查 (LLM 对比 docs/)
   │  不通过 → 生成修复指令 → 回到 ②（最多 3 次）
   ▼
④ git push → orchestrator 本地自动测试
   │  post-receive hook 触发 orchestrator 本地执行
   │  pytest 单元测试 + 云端API多模型Judge 业务评估 (GPT-4.1/Qwen-Max/DeepSeek-V3)
   │  测试报告本地直读，零网络延迟
   ▼
⑤ 判定
   ├─ 🟢 通过 → 取下一个任务 → 回到 ②
   ├─ 🔴 失败 → 生成修复指令 → 回到 ②（最多 3 次重试）
   └─ ⚠️ 3 次仍失败 → 钉钉通知人工介入
```

- **单个任务周期**: 5-15 分钟
- **每天可完成**: 50-100 个独立编码任务
- **运行方式**: 7×24 持续运行（白天高优先级任务，夜间重构/补测试）

13.4 接口契约：5 台机器的同步枢纽

5 台机器并行编码的关键是**接口对齐**。解决方案不变：**contracts/ 目录作为单一真相源**。

```
ai-crm/
├── contracts/                  ← 接口契约目录（你维护）
│   ├── agent-api.yaml          ← Agent 引擎对外接口定义
│   ├── crm-api.yaml            ← CRM 后端对外接口定义
│   ├── agent-tools.yaml        ← Agent Tool Calling Schema
│   ├── event-bus.yaml          ← Redis Stream 事件格式
│   └── db-schema.sql           ← 数据库表结构（单一真相源）
├── orchestrator/               ← Orchestrator 薄调度层（orchestrator 专用服务器）
├── agent/                      ← 4090 aider 自动编码
├── crm/                        ← mac_min_8T aider 自动编码
├── frontend/                   ← mac_min_8T aider 自动编码
├── deploy/                     ← gateway + data_center aider 自动编码
├── scripts/                    ← data_center aider 自动编码
├── tests/                      ← orchestrator 本地自动执行
└── docs/                       ← 你维护
```

自动化流程中的契约使用：

```
1. 你在 contracts/ 中定义或更新接口契约
   → git push

2. Orchestrator 检测到 contracts/ 变更
   → 自动为相关机器生成对齐编码任务
   → 例如: contracts/crm-api.yaml 新增接口 → W2 自动生成对应 API

3. aider 编码时自动加载 contracts/ 文件作为上下文
   → 保证生成的代码与契约签名一致

4. AutoReviewer Layer 2 自动对比代码与契约
   → 不一致自动修正
```

13.5 核心工具链

工具	用途	部署位置
aider	AI 编码 CLI，通过 OpenAI 兼容 API 连接 Claude Opus 4.6 自动生成代码并 commit	4090, mac_min_8T, gateway, data_center, orchestrator
vLLM 实例 A	Qwen3-30B-A3B 推理服务，**生产 Agent 推理专用**（不再承担编码推理）	4090 (2×4090)
~~vLLM 实例 B~~	~~已移除~~（AutoReview/DocParser 改用云端 API，orchestrator 2×4090 GPU 释放）	—
~~vLLM 实例 C~~	~~已移除~~（aliyun-agent 已从架构中移除，预发布验收由 orchestrator 承担，Agent 推理统一使用 vLLM A）	—
Orchestrator	薄调度层脚本（~200 行 shell/Python）：解析任务列表 → SSH 分发 aider 指令 → 收集 exit code → 通知。不重复实现 aider 已有的编码/测试/修复循环	orchestrator (专用服务器)
Git + hook	代码版本管理 + 推送自动触发测试	mac_min_8T (裸仓库), orchestrator (hook 触发本地测试)
pytest	自动化测试，编码完成后本地触发，零网络延迟	orchestrator (本地执行)
云端多模型 Judge	GPT-4.1/Qwen-Max/DeepSeek-V3 三模型交叉评估	orchestrator 发起 API 调用
钉钉机器人	进度通知、异常告警、每日/Sprint 摘要	gateway

Claude Opus 4.6 编码接入（当前实施值）：

```
export AIDER_MODEL="openai/claude-opus-4-6"
export OPENAI_API_BASE="${OPENAI_API_BASE}"   # 见 .env
export OPENAI_API_KEY="${OPENAI_API_KEY}"     # 见 .env
```

13.6 Git 工作流：全自动不冲突

```
        Git 裸仓库 (mac_min_8T:/opt/git/ai-crm.git)
                          │
      ┌──────────────────┼──────────────────┐
      │                  │                  │
  ┌──┴──────┐   ┌──────┴──────┐   ┌──────┴──────┐
  │ 4090    │   │ mac_min_8T  │   │ gateway /   │   ┌──────────┐
  │ agent/  │   │ crm/        │   │ data_center │   │orchestrator│
  │ 目录为主 │   │ frontend/   │   │ deploy/     │   │orchestrator│
  └─────────┘   │ 目录为主    │   │ 目录为主    │   │/ 目录为主  │
                └─────────────┘   └─────────────┘   └──────────┘
                          │
          Orchestrator 控制 push 顺序（避免冲突）
                          │
                post-receive hook
                          │
                  ┌──────┴──────┐
                  │ orchestrator     │
                  │ 本地 pytest 执行 │
                  │ 报告本地直读     │
                  └────────────────┘
```

不冲突的秘诀：每台机器修改不同目录 + Orchestrator 串行 push

机器	主要修改目录	其他机器是否会改
orchestrator	`orchestrator/`	否
4090	`agent/` , `contracts/agent-*.yaml`	否
mac_min_8T	`crm/` , `frontend/` , `contracts/crm-*.yaml`	否
gateway	`deploy/nginx/` , `deploy/dingtalk/`	否
data_center	`deploy/monitoring/` , `scripts/`	否
你手动	`contracts/` , `docs/`	契约和文档由你统一推送

13.7 自动 Review 机制（三层检查）

人工 Review 被 LLM 驱动的自动三层 Review 完全替代：

```
Layer 1: 静态检查 (0 成本, 秒级)
├── python -m py_compile  ← 语法正确性
├── ruff check            ← 代码规范
└── 结果: 有错 → 直接生成修复指令

Layer 2: 契约对齐检查 (LLM 驱动, ~30s)
├── 对比代码与 contracts/*.yaml 的接口签名
└── 结果: 不一致 → 生成对齐修复指令

Layer 3: 设计符合度检查 (LLM 驱动, ~60s)
├── 对比代码与 docs/04-系统概要设计.md 的设计意图
├── 评分: 功能完整性 / 接口正确性 / 错误处理 / 代码质量 / 可运行性
└── 结果: < 4.0/5.0 分 → 生成具体修改建议 → 重试

三层全通过 → git push → orchestrator 本地自动测试
```

13.8 监控与报告体系

```
Level 1: 实时日志 (每批次)
└── Orchestrator 日志 + Redis 状态

Level 2: 每日摘要 (18:00 自动推送钉钉)
├── 完成/失败/重试任务数
├── 代码行数变化
├── 测试通过率趋势
└── 云端多模型Judge 加权均分

Level 3: Sprint 报告 (Sprint 结束时推送钉钉)
├── 里程碑达成情况
├── 全量测试通过率
└── 遗留问题 + 下一 Sprint 计划
```

13.9 人工介入触发条件

流水线设计为**全自动**，仅以下情况通过钉钉通知人工介入：

#	触发条件	你需要做什么
1	编码任务连续 3 次 Review 失败	检查设计文档/契约是否有歧义，修改后 push
2	多模型Judge 加权评分 < 3.0	检查 Prompt 设计或业务逻辑，查看各Judge评分明细
3	硬件故障 (SSH不通/GPU异常)	物理排查/重启机器
4	Sprint 里程碑检查点	查看报告，确认进入下一 Sprint
5	设计文档与代码偏差 > 30%	更新设计文档或回退代码

> **大部分情况下你只需在钉钉回复「继续」或「暂停」。**

13.10 你的角色：方案设计者

```
┌──────────────────────────────────────────────────────────────┐
│  旧角色 (v1.0): 指挥官                                        │
│  ├── 打开 5 个 VS Code 窗口                                   │
│  ├── 逐窗口给 Copilot 下达指令                                │
│  ├── 巡回 Review 每个窗口的代码                               │
│  ├── 手动 git push/pull 同步                                  │
│  └── 瓶颈: 你的 Review 速度限制了整体产出                     │
│                                                               │
│  新角色 (v2.0): 方案设计者                                    │
│  ├── 编写/更新 docs/ 中的设计文档                             │
│  ├── 编写/更新 contracts/ 中的接口契约                        │
│  ├── git push 更新文档                                        │
│  ├── 查看钉钉每日摘要 (5 分钟)                                │
│  ├── 处理少量升级异常 (0-3 次/天)                             │
│  └── Sprint 结束时确认里程碑                                  │
│                                                               │
│  你的时间分配:                                                │
│  ├── 设计文档编写/更新: 60%                                   │
│  ├── 查看报告+处理异常: 20%                                   │
│  └── 规划+思考: 20%                                           │
│                                                               │
│  你不再需要:                                                  │
│  ├── ❌ 打开 VS Code 窗口                                     │
│  ├── ❌ 给 AI 逐条下达编码指令                                │
│  ├── ❌ 巡回 Review 代码                                      │
│  ├── ❌ 手动 git push/pull                                    │
│  └── ❌ 关注每个编码细节                                      │
└──────────────────────────────────────────────────────────────┘
```

13.11 开发效率估算

全自动化流水线 vs 旧模式的效率对比：

维度	传统单人开发	Copilot 5窗口并行 (v1.0)	全自动流水线 (v2.0)
Agent Prompt + Tool (10 个)	40h	10h	6h (aider 并行+自动重试)
CRM API (8 资源 × CRUD)	16h	4h	2h (aider 根据契约自动生成)
数据库模型 + 迁移 (15 张表)	22h	4.5h	2h
Vue3 页面 (6 页)	24h	9h	5h
Nginx + 钉钉集成	8h	2h	1h
监控 + 备份 + Celery	8h	2h	1h
联调 + 调试	40h	12h	5h (自动Review+自动测试大幅减少联调)
Review	—	15h (人工巡回)	0h (三层自动Review)
测试 + 评估	20h	3h	2h (全自动)
总计	**178h**	**~61.5h**	**~24h (约 3 个工作日)**

> **全自动模式的核心优势**：
>
> 1. **消除人工瓶颈** — Review 不再是串行瓶颈，三层自动检查比人工更严格
> 2. **7×24 运行** — 不受人类工作时间限制，夜间也在编码
> 3. **自动重试** — 编码失败自动修复，不需等人排查

> 4. **一致性** — 所有代码都经过统一的静态+契约+设计三层检查
>
> 一期 8 周时间非常充裕，关键路径仍是 **Agent Prompt 效果调优**。

14. 待确认清单

#	问题	影响范围	紧迫度
1	分期范围是否确认？一期还需加入什么？	开发范围	高
2	钉钉应用已注册？有 AppKey/AppSecret?	钉钉集成	高
3	LLM 模型：Qwen3-30B-A3B 还是升级 Qwen3.5-35B-A3B?	AI 推理	中
4	《AI神经系统组织-V2》和"十二条宪法"可提供?	Agent 治理	中
5	是否有现成的产品资料/话术/FAQ 可导入知识库?	知识库	中
6	ERP U9 接口文档是否可提供 (二期需要)?	系统集成	低
7	sirius_brain 的 NLU Prompt 和 Skill YAML 是否直接复用?	开发效率	中

附录

A. 文档规划

文档	状态	说明
01-基础需求-sirus项目需求.md	✅ 已提供	基础 CRM 需求
02-智能扩展需求.md	✅ 已提供	AI 智能扩展需求
03-硬件清单.md	✅ 完成	硬件清单
04-系统概要设计 (本文档)	✅ v0.4	AI-Native 架构设计 (§13 全自动化流水线)
05-测试方案与计划.md	✅ v0.2	AI-Native 测试方案、Copilot 驱动的开发测试一体化、L20 测试机
06-总体开发计划.md	✅ v2.0	里程碑、关键路径、风险、验收标准
07-Sprint任务卡.md	✅ v2.0	4 个 Sprint 的逐日任务卡（每机器每天的自动化任务）
08-自动化开发流水线.md	✅ v2.0	Orchestrator 调度、aider 编码、三层自动 Review、Git 自动化、监控报告
09-测试机搭建计划.md	⚠️ 已废弃	~~aliyun-agent 预发布环境搭建~~ — aliyun-agent 已移除，预发布验收由 orchestrator 承担
10-Agent 详细设计	🔜 待编写	Prompt 设计、Tool 定义、Agent 间协议
11-数据库设计	🔜 待编写	详细表结构、索引、迁移脚本
12-钉钉集成方案	🔜 待编写	机器人/卡片/审批对接细节
13-部署方案	🔜 待编写	Docker Compose、运维脚本、监控配置

B. 术语表

基础概念

术语	说明
CRM	Customer Relationship Management，客户关系管理系统。帮助企业管理客户信息、销售流程、售后服务的软件
CRUD	Create/Read/Update/Delete 的缩写，即数据的「增删改查」四种基本操作
API	Application Programming Interface，应用编程接口。软件之间相互通信的标准化方式，类似「插座接口」
RESTful API	一种 API 设计风格，用 HTTP 请求（GET/POST/PUT/DELETE）对应数据的查/增/改/删
SSO	Single Sign-On，单点登录。用一个账号（如钉钉）登录所有系统，不用重复输密码
JWT	JSON Web Token，一种登录凭证格式。用户登录后获得一个加密令牌，后续请求携带它证明身份
RBAC	Role-Based Access Control，基于角色的权限控制。不同角色（管理员/经理/销售）看到不同数据、执行不同操作
SDK	Software Development Kit，软件开发工具包。别人提供的现成代码库，拿来直接用

AI / 大模型相关

术语	说明
LLM	Large Language Model，大语言模型。能理解和生成自然语言的 AI 模型，如 ChatGPT、Qwen 等
Agent	AI 智能体。不只是回答问题，还能自主规划、调用工具、执行操作的 AI 程序
AI-Native	以 AI Agent 为系统核心（而非附加层）的架构模式。Agent 就是系统本身，而不是系统上装了一个 AI 插件
Supervisor Agent	总调度 Agent，接收用户消息后理解意图、分解任务、编排执行，相当于 Agent 团队的「项目经理」
Tool Calling	工具调用。LLM 在对话中识别出需要执行的操作，自动调用对应的函数（如查数据库、发消息），而不只是生成文字
Prompt	提示词。给 LLM 的指令和上下文，决定了 LLM 如何思考和回答。设计好的 Prompt 是 Agent 能力的关键
Token	LLM 处理文本的最小单位，约等于 1-2 个中文字或 3-4 个英文字母。LLM 的输入输出都按 Token 计量
MoE	Mixture of Experts，混合专家架构。模型虽然总参数很大（30B），但每次只激活一小部分（3B），所以推理速度快、显存占用低
vLLM	一个高性能 LLM 推理引擎，能高效利用 GPU 同时处理多个请求，是我们用来运行 Qwen 模型的软件
Continuous Batching	连续批处理。vLLM 的核心技术，请求随到随处理，不用等凑满一批，大幅降低延迟
PagedAttention	vLLM 的显存管理技术，像操作系统管理内存一样管理 GPU 显存，让多个请求共享显存，提高并发能力
Prefix Caching	前缀缓存。多个请求如果开头相同（如同样的系统提示词），GPU 里只计算一次，后续复用，节省时间
Tensor Parallel	张量并行。把一个模型拆到多张 GPU 卡上同时计算，加快推理速度。我们用 2 张 4090 并行
Embedding	向量嵌入。把文本转换成一串数字（向量），数字相近的文本含义也相近。用于知识库的语义搜索
向量数据库	专门存储和检索向量的数据库。输入一段文字，能找到含义最相近的其他内容
pgvector	PostgreSQL 的向量搜索扩展插件，让普通数据库也能做向量检索，不用额外装一个独立的向量数据库
RAG	Retrieval-Augmented Generation，检索增强生成。Agent 先从知识库检索相关内容，再结合检索结果生成回答，提高准确性
NLU	Natural Language Understanding，自然语言理解。让 AI 理解人说的话是什么意思（识别意图和关键信息）

术语	说明
SSE	Server-Sent Events，服务器推送事件。一种流式传输技术，让 Agent 的回答像打字一样逐字显示，而不是等全部生成完才一次返回

架构相关

术语	说明
Supervisor	见 Supervisor Agent
结构化记忆	Agent 的长期记忆，存在 PostgreSQL 里。就是 CRM 的业务数据：客户、商机、合同等
语义记忆	Agent 的知识库，通过向量存储。存放：赢单经验、产品知识、行业知识等，支持按含义检索
工作记忆	Agent 的短期记忆，存在 Redis 里。包括：当前对话上下文、待确认的操作、正在执行的任务状态
事件总线	Agent 的神经网络。所有业务操作产生事件 → 事件总线广播 → 关心这个事件的 Agent 自动触发。用 Redis Stream 实现
审计事实流	不可变的操作日志。记录「谁在什么时间对什么数据做了什么操作」，只增不改不删，用于审计和回溯
事件驱动	一种架构模式：系统中发生的事情（如创建线索）产生「事件」，其他模块订阅感兴趣的事件自动响应，而不是互相直接调用
降级	当 AI 服务出故障时，系统自动切换到无 AI 的基本模式继续运行（如用 Web 表单做 CRUD），保证业务不中断
幂等	同一个操作执行一次和执行多次效果相同。比如「把商机阶段改为方案评审」执行两次，结果应该和执行一次一样

技术组件

术语	说明
FastAPI	Python 的 Web 框架，用于构建 API 服务。特点是高性能、自动生成接口文档、支持异步
PostgreSQL	开源关系型数据库，功能强大。我们用它存所有 CRM 数据和向量数据
Redis	开源的内存数据库，读写极快。用于缓存、对话上下文存储、事件总线、任务队列
Redis Stream	Redis 的消息队列功能，用来实现事件总线。生产者发布事件，消费者（Agent）订阅和处理事件
SQLAlchemy	Python 的 ORM 框架。ORM = 对象关系映射，让代码用 Python 对象操作数据库，不用直接写 SQL
Alembic	数据库迁移工具。当数据库表结构需要修改时（如加字段），用它管理变更记录并自动执行
Celery	Python 的异步任务队列。把耗时任务（如批量评分、报告生成）扔到后台执行，不阻塞用户操作
Docker	容器化技术。把应用和它的依赖打包在一起，确保在任何机器上都能跑起来，一键部署
Docker Compose	Docker 的编排工具。一个配置文件定义好所有服务（数据库、Redis、后端、Agent 引擎），一条命令全部启动
Nginx	反向代理服务器，放在最前面接收所有外部请求，然后转发到内部各个服务。同时负责 SSL 加密和限流
Prometheus	开源监控系统，定时采集各服务的运行指标（CPU、内存、响应时间、错误率等）
Grafana	可视化监控面板，把 Prometheus 采集的数据画成图表，一眼看到系统运行状态
Jinja2	Python 的模板引擎。用模板+变量生成文本，我们用它管理 Agent 的 Prompt 模板
Vue 3	JavaScript 前端框架，用于构建 Web 界面。国内主流，生态成熟
Element Plus	基于 Vue 3 的 UI 组件库，提供现成的表格、表单、按钮等企业级界面组件
ECharts	JavaScript 图表库，用于画漏斗图、趋势图、饼图等数据可视化图表
Vite	前端构建工具，把 Vue 代码打包成浏览器能运行的文件，打包速度很快
Loguru	Python 的日志库，比内置 logging 更好用，支持结构化 JSON 格式日志

业务术语

术语	说明
线索 (Lead)	潜在的销售机会信息，尚未确认是否有真实需求。比如展会扫码留下联系方式的人
客户 (Customer/Account)	已确认有合作意向或已成交的企业/个人
联系人 (Contact)	客户企业中的具体对接人，一个客户可能有多个联系人
商机 (Opportunity)	有明确采购意向的销售机会，有预期金额和成交时间。从线索转化而来
漏斗 (Funnel)	销售漏斗，把销售流程可视化为漏斗形状：大量线索 → 较少客户 → 少量商机 → 成交。 每层的转化率是核心指标
赢率	商机最终成交的概率预测，由 Agent 基于多维度数据计算
客户画像	对一个客户的全方位描述：基本信息、交易历史、互动记录、偏好特征、健康度等
360 视图	客户 360 度视图，把客户相关的所有信息（商机、合同、活动、工单等）汇聚在一个页面展示
DSO	Days Sales Outstanding，应收账款周转天数。衡量从销售到收到钱平均需要多少天
CLV	Customer Lifetime Value，客户生命周期价值。预测一个客户在整个合作期间能带来多少收入
决策链	客户内部的采购决策关系：谁是最终决策者、谁是影响者、谁是使用者。搞清决策链是赢单关键
线索池	未分配给具体销售人员的线索集合，销售可以从池中领取或由系统自动分配